

Loan Default Risk Prediction

Deliverable 1: Problem Definition & Data Foundations

23L-2524

23L-2560

Contents

1	Phase 1: Problem Definition	2
1.1	Problem Statement	2
1.2	Target Audience	2
1.3	AI Justification	2
2	Phase 2: Data Acquisition & Preprocessing	3
2.1	Dataset Overview	3
2.2	Data Cleaning	4
2.2.1	Missing Values	4
2.2.2	Duplicate Rows	4
2.3	Exploratory Data Analysis	4
2.3.1	Target Class Distribution	4
2.3.2	Feature Distributions	5
2.3.3	Correlation Heatmap	5
2.3.4	Feature Distributions by Target Class	6
2.3.5	Age Distribution by Target	6
2.3.6	Default Rate by Age Group	7
2.3.7	Default Rate by Credit Utilization	7
2.4	Outlier Detection and Treatment	8
2.4.1	Identified Issues	8
2.5	Normalization and Scaling	9
2.5.1	Encoding	9
2.5.2	Scaler Comparison	9
3	Repository Structure	10
4	Pipeline Execution Order	10

1 Phase 1: Problem Definition

1.1 Problem Statement

Financial institutions process thousands of loan applications daily. Manual credit risk assessment is slow, inconsistent, and prone to human bias. A single bad lending decision can result in significant financial loss, while overly conservative decisions reject creditworthy applicants.

This project builds a machine learning application that predicts loan default risk using applicant financial and credit behavior data. The system accepts new applicant information as input, analyzes key financial and credit factors, and returns a default-risk prediction classified into **Low**, **Medium**, or **High** risk categories. Each prediction includes explainable insights showing the specific factors that influenced the decision, enabling loan officers to make faster, more consistent, and transparent lending decisions.

1.2 Target Audience

The primary users of this tool are:

- **Loan officers and credit analysts** at banks and lending institutions who need a fast, data-driven second opinion on applicant risk.
- **Risk management teams** who need consistent, auditable criteria for loan approval decisions.
- **Fintech companies** offering automated lending products that require real-time risk scoring with regulatory-compliant explanations.

1.3 AI Justification

Traditional rule-based credit scoring relies on fixed thresholds (e.g., “reject if debt ratio > 0.5 ”). This approach fails to capture the complex, non-linear interactions between financial variables that predict default. For example:

- A high debt ratio may be acceptable for a young borrower with zero delinquencies, but dangerous for an older borrower with past late payments.
- Credit utilization above 75% combined with multiple recent inquiries signals very different risk than the same utilization with a stable credit history.

Machine learning models can learn these multi-variable interaction patterns from historical data, delivering more accurate and nuanced risk assessments than static rules. Specifically:

- **Non-linearity:** Tree-based models (Random Forest, XGBoost) capture threshold effects and feature interactions without manual rule writing.
- **Scalability:** A trained model scores thousands of applicants per second, compared to minutes per application for manual review.
- **Explainability:** SHAP and LIME provide per-prediction explanations, satisfying regulatory requirements for transparent lending decisions.

2 Phase 2: Data Acquisition & Preprocessing

2.1 Dataset Overview

The dataset used is **Give Me Some Credit** from Kaggle, a widely-used benchmark for credit risk modeling. It contains historical borrower data with a binary target variable indicating whether the borrower experienced serious delinquency within 2 years.

Property	Value
Source	Kaggle (Give Me Some Credit)
Training rows	150,000
Test rows	Separate file (cs-test.csv)
Features	10 numerical
Target	SeriousDlqin2yrs (0 = No Default, 1 = Default)
Class balance	93.3% No Default / 6.7% Default

Table 1: Dataset summary

Original Name	Mapped Name	Description
RevolvingUtilizationOfUnsecuredLines	RevolvingUtilization	Credit card balance / limit
age	Age	Borrower age in years
NumberOfTime30-59DaysPastDueNotWorse	Times30_59Late	30–59 day late payments
DebtRatio	DebtRatio	Monthly debt / income
MonthlyIncome	MonthlyIncome	Gross monthly income
NumberOfOpenCreditLinesAndLoans	OpenCreditLines	Open credit lines count
NumberOfTimes90DaysLate	Times90Late	90+ day late payments
NumberRealEstateLoansOrLines	RealEstateLines	Real estate loan count
NumberOfTime60-89DaysPastDueNotWorse	Times60_89Late	60–89 day late payments
NumberOfDependents	Dependents	Number of dependents

Table 2: Feature descriptions and column mapping

2.2 Data Cleaning

2.2.1 Missing Values

Two columns had missing values:

Column	Missing	%	Action
MonthlyIncome	29,731	19.82%	Filled with median (5,400)
Dependents	3,924	2.62%	Filled with median (0)
Age	1	<0.01%	Age = 0 replaced with median (52)

Table 3: Missing value handling

Why median? Both MonthlyIncome (skewness = 114.04) and Dependents (skewness = 1.59) are heavily right-skewed. Using the mean would overestimate typical values because it gets pulled up by extreme outliers. The median is robust to this skew.

2.2.2 Duplicate Rows

767 duplicate rows were found and removed, reducing the dataset from 150,000 to 149,233 rows.

2.3 Exploratory Data Analysis

2.3.1 Target Class Distribution

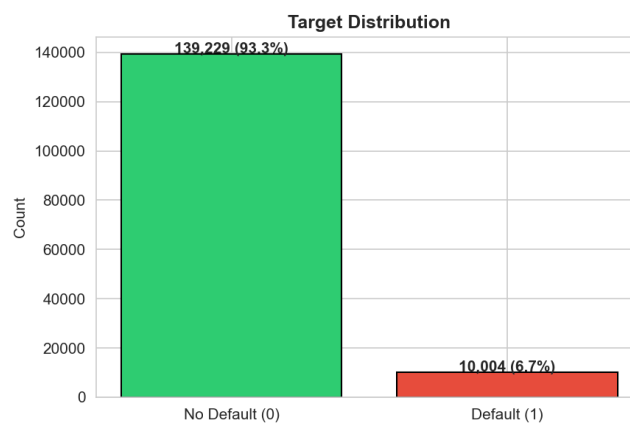


Figure 1: Target class distribution

The dataset is heavily imbalanced — 93.3% of borrowers did not default while only 6.7% did. This means any model trained without addressing this imbalance would achieve 93% accuracy by simply predicting “No Default” for everyone, which is useless in practice. This will be handled via SMOTE in the modeling phase.

2.3.2 Feature Distributions

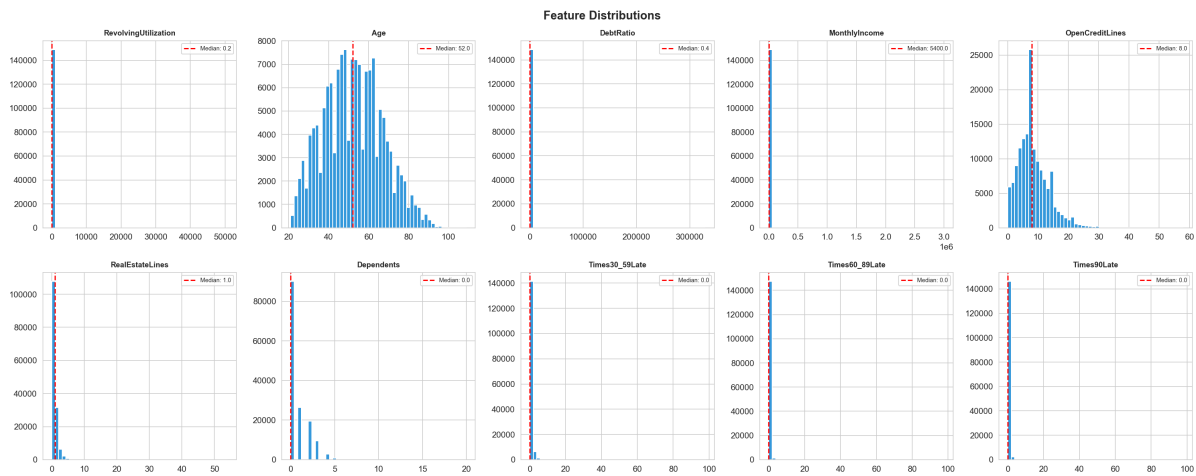


Figure 2: Feature distributions with median lines

Most features are right-skewed with long tails. RevolvingUtilization and DebtRatio have extreme maximum values (50,708 and 329,664 respectively) while their medians are near 0.15 and 0.37 — clear evidence of severe outliers. The late payment features (Times30_59Late, Times60_89Late, Times90Late) show that the vast majority of borrowers have zero late payments, with a small group having extreme counts up to 98, which are sentinel codes rather than real values.

2.3.3 Correlation Heatmap

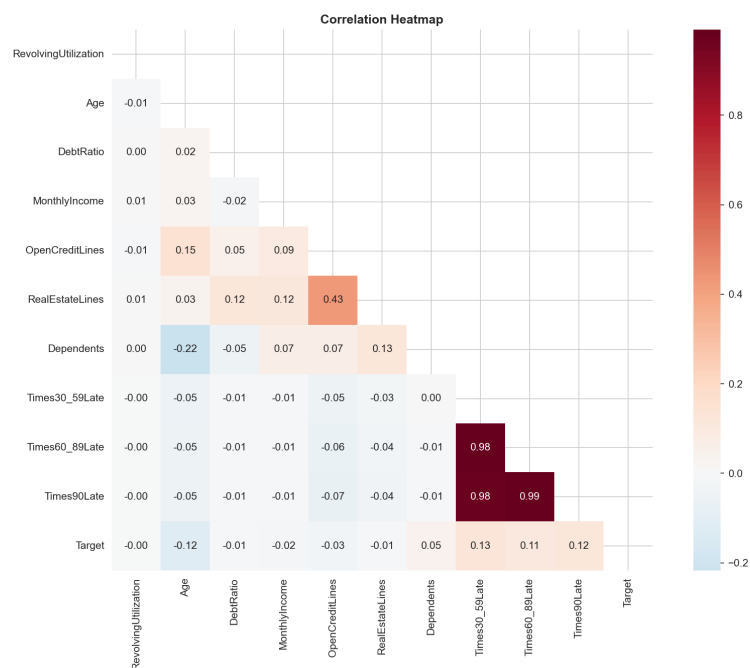


Figure 3: Feature correlation matrix

The three late payment features are highly correlated with each other (0.70–0.84), which is expected since a borrower who is 90+ days late has usually also been 30–59 and 60–89 days late. The strongest individual correlations with the target are RevolvingUtilization (0.28), Times30_59Late (0.25), and Times90Late (0.24). Age has a negative correlation with default (-0.12), meaning younger borrowers default more often.

2.3.4 Feature Distributions by Target Class

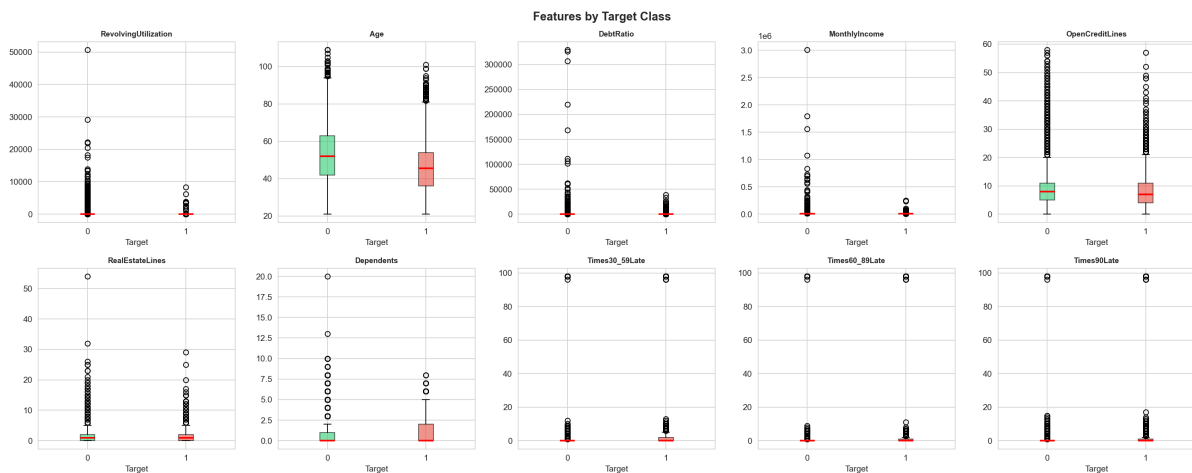


Figure 4: Boxplots of features grouped by target class

Defaulters (class 1) show noticeably higher RevolvingUtilization and more late payments compared to non-defaulters. The boxplots also expose outliers — both classes contain extreme values in DebtRatio and MonthlyIncome (dots far beyond the whiskers), confirming the need for outlier treatment before modeling.

2.3.5 Age Distribution by Target

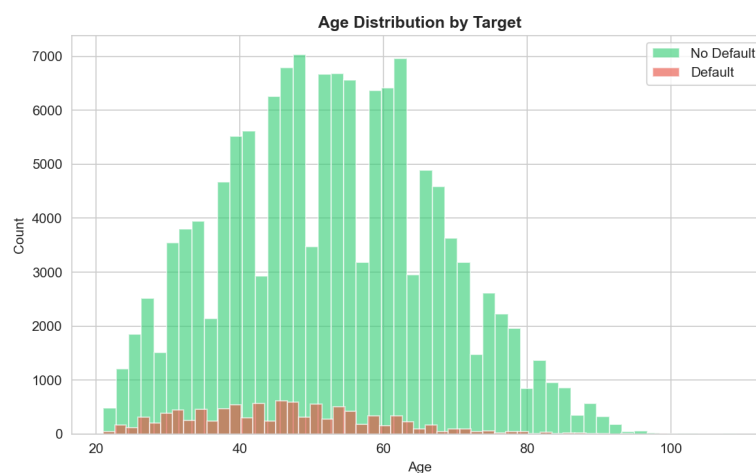


Figure 5: Age distribution by default status

The default group (red) is shifted toward younger ages compared to non-defaulters (green). Younger borrowers have a higher concentration in the default class, while the non-default group is more evenly spread and peaks later. This suggests age is a useful predictor of default risk.

2.3.6 Default Rate by Age Group

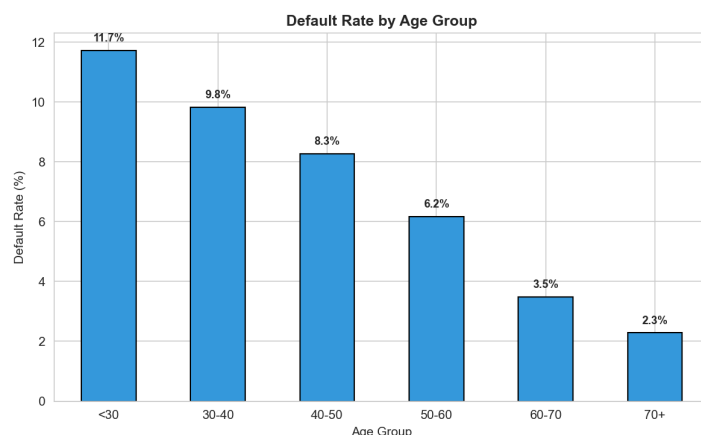


Figure 6: Default rate by age group

Default rate decreases steadily with age — from 11.7% for borrowers under 30 down to 2.3% for those over 70. This confirms a clear monotonic relationship: younger borrowers carry significantly higher default risk, possibly due to shorter credit histories and lower financial stability.

2.3.7 Default Rate by Credit Utilization

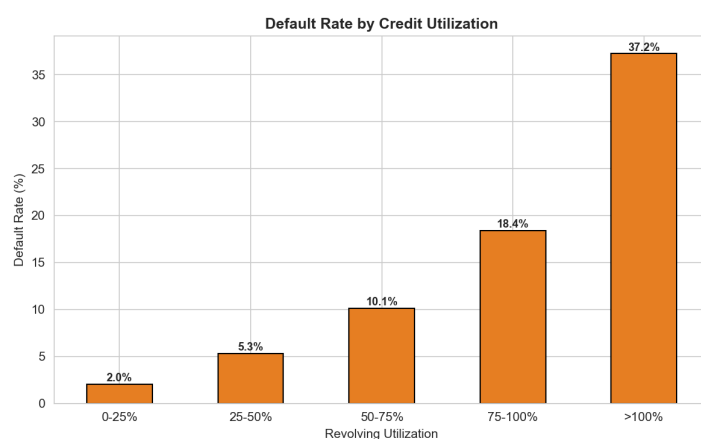


Figure 7: Default rate by revolving credit utilization

Higher credit utilization directly maps to higher default risk. Borrowers using less than 25% of their credit limit default at only 2.0%, while those exceeding 100% (overlimit) default at 37.2%. The >100% group also flags potential data quality issues — utilization

above 1.0 may indicate data errors or extreme financial distress, both of which justify outlier capping.

2.4 Outlier Detection and Treatment

Outliers were first identified using the IQR method, then treated using capping (win-sorization) at the 99th percentile. Capping was chosen over removal to preserve data volume.

2.4.1 Identified Issues

- **Sentinel codes:** The late payment columns contain values 96 and 98, which are codes (not real counts). 216 rows affected per column — capped at 20.
- **RevolvingUtilization:** Max value of 50,708 vs median of 0.15 — clear data errors. Capped at 99th percentile (1.09).
- **DebtRatio:** Max value of 329,664 vs median of 0.37 — capped at 99th percentile (4,988).
- **MonthlyIncome:** Max value of 3,008,750 vs median of 5,400 — capped at 99th percentile (23,098).
- **OpenCreditLines:** Capped at 99th percentile (24).
- **RealEstateLines:** Capped at 99th percentile (4).

Feature	Min	Max (After)	Median
RevolvingUtilization	0.00	1.09	0.15
Age	21.00	109.00	52.00
DebtRatio	0.00	4,988.04	0.37
MonthlyIncome	0.00	23,097.76	5,400.00
OpenCreditLines	0.00	24.00	8.00
RealEstateLines	0.00	4.00	1.00
Dependents	0.00	20.00	0.00
Times30_59Late	0.00	20.00	0.00
Times60_89Late	0.00	20.00	0.00
Times90Late	0.00	20.00	0.00

Table 4: Feature ranges after outlier treatment

2.5 Normalization and Scaling

2.5.1 Encoding

All features in this dataset are numerical. No categorical encoding (one-hot or label encoding) was required. The target variable is already binary (0/1).

2.5.2 Scaler Comparison

Three scalers were compared on the MonthlyIncome feature:

Scaler	Min	Max	Mean	Std
StandardScaler	-1.60	4.41	0.00	1.00
MinMaxScaler	0.00	1.00	0.27	0.17
RobustScaler	-1.54	5.04	0.21	1.10

Table 5: Scaler comparison on MonthlyIncome

RobustScaler was selected because it uses the median and interquartile range (IQR) instead of the mean and standard deviation. Since several features (DebtRatio, RevolvingUtilization) still have skewed distributions even after outlier capping, RobustScaler is less sensitive to remaining extreme values than StandardScaler or MinMaxScaler. The scaler was fitted on the training data and saved as a pickle file for later use on the test set.

3 Repository Structure

```
CreditShield-Loan-Default-Risk-Predictor/
|-- README.md
|-- .gitignore
|-- requirements.txt
|-- data_scripts/
|   |-- preprocess.py
|   |-- eda.py
|   |-- outliers_handling.py
|   |-- scaling_normalization.py
|-- figures/
|   |-- 01_target_distribution.png
|   |-- 02_feature_distributions.png
|   |-- 03_correlation_heatmap.png
|   |-- 04_boxplots_by_target.png
|   |-- 05_age_by_target.png
|   |-- 06_default_rate_by_age.png
|   |-- 07_default_rate_by_utilization.png
|-- outputs/
|   |-- cleaned.csv
|   |-- outliers_handled.csv
|   |-- scaled_dataset.csv
|   |-- robust_scaler.pkl
|-- reports/
|   |-- deliverable1_report.tex
|   |-- deliverable1_report.pdf
```

4 Pipeline Execution Order

1. `preprocess.py` – Load data, map columns, impute missing values, remove duplicates
2. `eda.py` – Generate 7 EDA charts (saved to `figures/`)
3. `outliers_handling.py` – Detect and handle outliers
4. `scaling_normalization.py` – Encoding check, comparing different scalers , apply RobustScaler